F^2-NeRF: Fast Neural Radiance Field Training with Free Camera Trajectories

Peng Wang[1,2*] Yuan Liu[1*] Zhaoxi Chen[2] Lingjie Liu[3]
Ziwei Liu[2] Taku Komura[1] Christian Theobalt[3] Wenping Wang[4]

[1] The University of Hong Kong [2] S-Lab, Nanyang Technological University
[3] Max Planck Institute for Informatics [4] Texas A&M University

Abstract

*This paper presents a novel grid-based NeRF called **F^2-NeRF (Fast-Free-NeRF)** for novel view synthesis, which enables arbitrary input camera trajectories and only costs a few minutes for training. Existing fast grid-based NeRF training frameworks, like Instant-NGP, Plenoxels, DVGO, or TensoRF, are mainly designed for bounded scenes and rely on space warping to handle unbounded scenes. Existing two widely-used space-warping methods are only designed for the forward-facing trajectory or the 360° object-centric trajectory but cannot process arbitrary trajectories. In this paper, we delve deep into the mechanism of space warping to handle unbounded scenes. Based on our analysis, we further propose a novel space-warping method called perspective warping, which allows us to handle arbitrary trajectories in the grid-based NeRF framework. Extensive experiments demonstrate that F^2-NeRF is able to use the same perspective warping to render high-quality images on two standard datasets and a new free trajectory dataset collected by us. Project page: totoro97.github.io/projects/f2-nerf.*

1. Introduction

The research progress of novel view synthesis has advanced drastically in recent years since the emergence of the Neural Radiance Field (NeRF) [24, 42]. Once the training is done, NeRF is able to render high-quality images from novel camera poses. The key idea of NeRF is to represent the scene as a density field and a radiance field encoded by Multi-layer Perceptron (MLP) networks, and optimize the MLP networks with the differentiable volume rendering technique. Though NeRF is able to achieve photo-realistic rendering results, training a NeRF takes hours or days due to the slow optimization of deep neural networks, which limits its application scopes.

Recent works demonstrate that grid-based methods, such as Plenoxels [57], DVGO [38], TensoRF [6], and Instant-NGP [25], enable fast training a NeRF within a few minutes. However, the memory consumption of such grid-based rep-

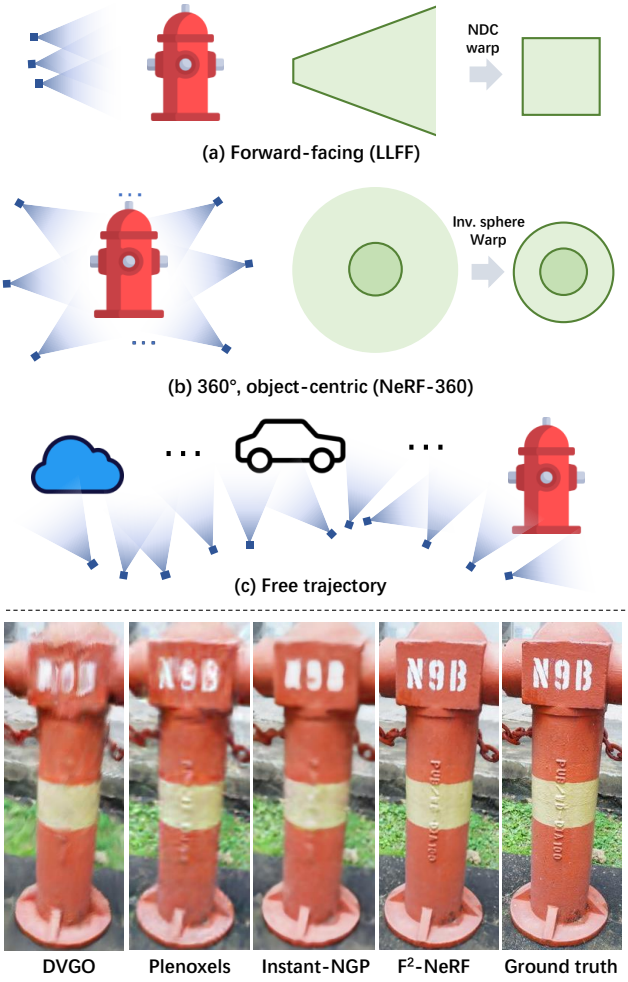

Figure 1. Top: (a) Forward-facing camera trajectory. (b) 360° object-centric camera trajectory. (c) Free camera trajectory. In (c), the camera trajectory is long and contains multiple foreground objects, which is extremely challenging. Bottom: Rendered images of the state-of-the-art fast NeRF training methods and F^2-NeRF on a scene with a free trajectory.

*Equal contribution.

resentations grows in cubic order with the size of the scene. Though various techniques, such as voxel pruning [38, 57], tensor decomposition [6] or hash indexing [25], are proposed to reduce the memory consumption, these methods still can only process bounded scenes when grids are built in the original Euclidean space.

To represent unbounded scenes, a commonly-adopted strategy is to use a space-warping method that maps an unbounded space to a bounded space [3, 24, 61]. There are typically two kinds of warping functions. (1) For forward-facing scenes (Fig. 1 (a)), the Normalized Device Coordinate (NDC) warping is used to map an infinitely-far view frustum to a bounded box by squashing the space along the z-axis [24]; (2) For 360° object-centric unbounded scenes (Fig. 1 (b)), the inverse-sphere warping can be used to map an infinitely large space to a bounded sphere by the sphere inversion transformation [3, 61]. Nevertheless, these two warping methods assume special camera trajectory patterns and cannot handle arbitrary ones. In particular, when a trajectory is long and contains multiple objects of interest, called *free trajectories*, as shown in Fig. 1 (c), the quality of rendered images degrades severely.

The performance degradation on free trajectories is caused by the imbalanced allocation of spatial representation capacity. Specifically, when the trajectory is narrow and long, many regions in the scenes are empty and invisible to any input views. However, the grids of existing methods are regularly tiled in the whole scene, no matter whether the space is empty or not. Thus, much representation capacity is wasted on empty space. Although such wasting can be alleviated by using the progressive empty-voxel-pruning [38, 57], tensor decomposition [6] or hash indexing [25], it still causes blurred images due to limited GPU memory. Furthermore, in the visible spaces, multiple foreground objects in Fig. 1 (c) are observed with dense and near input views while background spaces are only covered by sparse and far input views. In this case, for the optimal use of the spatial representation of the grid, dense grids should be allocated for the foreground objects to preserve shape details and coarse grids should be put in background space. However, current grid-based methods allocate grids evenly in the space, causing the inefficient use of the representation capacity.

To address the above problems, we propose F^2-NeRF (Fast-Free-NeRF), the first fast NeRF training method that accommodates free camera trajectories for large, unbounded scenes. Built upon the framework of Instant-NGP [25], F^2-NeRF can efficiently be trained on unbounded scenes with diverse camera trajectories and maintains the fast convergence speed of the hash-grid representation.

In F^2-NeRF , we give the criterion on a proper warping function under an arbitrary camera configuration. Based on this criterion, we develop a general space-warping scheme called the *perspective warping* that is applicable to arbitrary camera trajectories. The key idea of perspective warping is to first represent the location of a 3D point $\mathbf{p}$ by the concatenation of the 2D coordinates of the projections of $\mathbf{p}$ in the input images and then map these 2D coordinates into a compact 3D subspace space using Principle Component Analysis (PCA) [51]. We empirically show that the proposed perspective warping is a generalization of the existing NDC warping [24] and the inverse sphere warping [3, 61] to arbitrary trajectories in a sense that the perspective warping is able to handle arbitrary trajectories while could automatically degenerate to these two warping functions in forward-facing scenes or 360° object-centric scenes. In order to implement the perspective warping in a grid-based NeRF framework, we further propose a space subdivision algorithm to adaptively use coarse grids for background regions and fine grids for foreground regions.

We conduct extensive experiments on the unbounded forward-facing dataset, the unbounded 360° object-centric dataset, and a new unbounded free trajectory dataset. The experiments show that F^2-NeRF uses the same perspective warping to render high-quality images on the three datasets with different trajectory patterns. On the new Free dataset with free camera trajectories, our method outperforms baseline grid-based NeRF methods,while only using $\sim$12 minutes on training on a 2080Ti GPU.

2. Related Works

Novel view synthesis. Novel view synthesis (NVS) aims to synthesize novel view images from input posed images. The NVS problem has been extensively studied with lumigraph [5, 13] and light field functions [8, 15] to directly interpolate input images. To improve the quality of the synthesized image, many methods resort to an explicit 3D reconstruction of the scene via meshes [9, 43, 47, 52], voxels [14, 20, 21, 34], point clouds [1, 54], depth maps [10, 31, 32, 45], and multi-plane images (MPI) [12, 16, 23, 36, 44, 63], and then synthesize novel images with the help of these 3D reconstructions. F^2-NeRF also aims to solve the NVS task but with a neural representation.

Neural scene representations. Since the emergence of NeRF [2, 24, 41], there have been intensive studies on neural representations for the tasks of novel view synthesis [24, 35, 42], relighting [4, 26, 60, 62], generalization to new scenes [7, 19, 37, 48, 50, 59], shape representation [22, 25, 39], and multi-view reconstruction [27, 49, 55, 56]. The representation can be either totally neural networks [11, 17, 24, 29, 33], or hybrid parametric encodings with space subdivisions [18, 22, 25, 39] for efficient training and inference. F^2-NeRF also subdivides the scene for flexible space warping and uses the hybrid neural scene representation [25] for fast training and high-quality rendering.

Fast NeRF training with space warping. Recent works show that the training of NeRF can be accelerated signifi-

cantly with grid-based representations [6, 25, 38, 57]. Instead of using a huge MLP network to predict the density and color, Plenoxels [57] directly store the density values and colors on a voxel grid. Instant-NGP [25], TensoRF [6] and DVGO [38] construct a feature grid and the density and compute the density and the color for a specific point from an interpolated feature vector using a tiny MLP network. However, these grids are regularly constructed in an axis-aligned manner and require additional space warping to represent unbounded scenes. There are two kinds of existing space warping functions, the NDC warping [24] for the forward-facing scenes and the inverse sphere warping [2, 3, 61] for the 360° object-centric scenes. Both of these warping functions cannot handle long and narrow trajectories. In F^2-NeRF , we propose a novel perspective warping to enable these fast grid-based methods to process arbitrary camera trajectories.

Large-scale Neural Radiance Fields. Recent works [40, 46, 53] managed to reconstruct the radiance field on a large-scale scene by decomposing the scene into blocks and separately training different NeRFs for different blocks. F^2-NeRF aims at small-scale scenes with arbitrary camera trajectories, which has the potential to serve as the backbone NeRF of a single block in large-scale NeRFs.

3. Our Approach

Given a set of images $\{\mathcal{I}_i\}$ with arbitrary but known poses in an unbounded scene, the goal of F^2-NeRF is to reconstruct a radiance field of the scene for the novel view synthesis task. In the following, we first give an overview of F^2-NeRF .

3.1. Overview

In order to build a grid-based neural representation in an unbounded scene, a space warping function $F(\mathbf{x})$ is introduced to warp the unbounded space to a bounded region. In Sec. 3.2, we first analyze the mechanism of space warping and propose a novel perspective warping method. The proposed perspective warping subdivides whole the space under consideration into small regions, as shown in Sec. 3.3. Then, on the warp space, we build a grid-based neural representation in Sec. 3.4. Based on the built representation, we adopt the volume rendering [24] to render novel-view images.

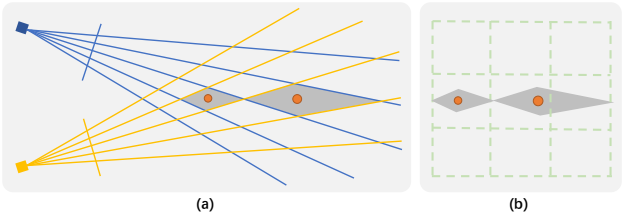

Figure 2. **A 2D example.** (a) The gray regions at the orange points align with the image resolutions. (b) Axis-aligned grids in the original Euclidean space are not aligned with the camera rays.

Volume rendering with perspective warping. In order to render the color $\hat{c}$ for a pixel, we first apply a novel point sampling strategy, called the *perspective sampling* in Sec. 3.5, to sample points $\mathbf{x}_i$ on the camera ray emitting from the pixel. Then, these sampled points are warped by the perspective warping to the warp space, and the density σ_i and the color c_i on sampled points are computed from the neural representation built on the warp space. Finally, we composite the colors to compute the pixel color $\hat{c}$ by

$$\hat{c} = \sum_i T_i \alpha_i c_i, \tag{1}$$

where $T_i = \prod_{j=0}^{i-1}(1-\alpha_j)$ is the accumulated transmittance and $\alpha_i = 1 - \exp(-\delta_i \sigma_i)$ is the opacity of the point.

3.2. Perspective warping

It has been demonstrated that space warping functions, such as the NDC warping [24] and the inverse sphere warping [3, 61], are effective for rendering unbounded scenes. In this section, we start with a 2D intuitive analysis of why a space warping method is effective.

2D analysis. First, let us consider a simple case in the 2D space as shown in Fig. 2 (a). In this setting, two 2D cameras project the points from 2D space onto their 1D image planes. Consider the two orange points in the figure. The gray rhombuses are the irregular grids formed by camera rays and the gray regions are the smallest distinguishable region due to the limited resolution of the two cameras. However, a vanilla grid-based representation consists of axis-aligned regular grids as shown in Fig. 2 (b), which is not aligned with the gray rhombuses. Moreover, such misalignment becomes more severe as the distance from the cameras increases. The key requirement for space warping is that we need to warp the original Euclidean space and build axis-aligned grids in the warp space so that these grids are aligned with the camera rays. Clearly, in this 2D case, a proper warping function $F(\mathbf{x}) : \mathbb{R}^2 \to \mathbb{R}^2$ can be constructed by $F(\mathbf{x}) = (C_1(\mathbf{x}), C_2(\mathbf{x}))$, where $C_1(\mathbf{x})$ and $C_2(\mathbf{x})$ denotes the 1D image coordinates of projecting $\mathbf{x}$ onto the camera 1 and camera 2 respectively. Then, the axis-aligned grids built on the $F(\mathbf{x})$ space will exactly align with camera rays.

Proper space warping. Based on the 2D analysis above, we define a proper space warping function as follows.

Definition 1 *Given a region S in the 3D Euclidean space and a set of cameras $\{C_i | i = 1, 2, ..., n_c\}$ which are visible to S, a warping function $F : \mathbb{R}^3 \to \mathbb{R}^3$ is called a* proper warping function*, if for any two points $\mathbf{x}_1, \mathbf{x}_2 \in S$, the distance between these two points in the warp space equals to the sum of distances between these two points on all visible cameras, i.e. $\|F(\mathbf{x}_1) - F(\mathbf{x}_2)\|_2^2 = \sum_i^n \|C_i(\mathbf{x}_1) - C_i(\mathbf{x}_2)\|_2^2$.*

Clearly, the warping function $F(\mathbf{x}) = (C_1(\mathbf{x}), C_2(\mathbf{x}))$ in the 2D toy example is a proper 2D warping function. Note that whether a warping function is proper or not is a local property, which only relates to the visible cameras.

3D perspective warping. Given the region S and the cameras C_i, we now would like to construct a proper 3D warping function $F(\mathbf{x}) : \mathbb{R}^3 \rightarrow \mathbb{R}^3$. We begin with an observation that the function $\mathbf{y} = G(\mathbf{x}) = [C_1(\mathbf{x}), ..., C_{n_c}(\mathbf{x})] : \mathbb{R}^3 \rightarrow \mathbb{R}^{2n_c}$ which maps the 3D point to its projected coordinates on all n cameras C_i is a proper warping function because this is a trivial construction based on the definition of a proper warping function. However, what we want to construct eventually is a function that maps a 3D space to a 3D space. Hence, we consider constructing an approximately proper warping function F with the following formulation.

Problem 1 *Let* $\{\mathbf{x}_j | j = 1, 2, ..., n_p\}$ *denote* n_p *evenly-sampled points in the local region* S *of the original Euclidean space, we want to find a projection matrix* $M \in \mathbb{R}^{3\times 2n_c}$ *that maps the coordinate* $\mathbf{y}_j = G(\mathbf{x}_j) \in \mathbb{R}^{2n_c}$ *to* $\mathbf{z}_j \in \mathbb{R}^3$ *by* $\mathbf{z}_j = M\mathbf{y}_j$*, so that* M *minimizes* $\sum_j^K \|M^\intercal \mathbf{z}_j - \mathbf{y}_j\|_2^2$

In comparison with Definition 1, Problem 1 makes two relaxations. First, we only consider the distances between sampled points $x_i \in S$ in Problem 1 while Definition 1 considers arbitrary point pairs. Second, we apply a projection matrix such that the distances between $\mathbf{y}_i$ are preserved as much as possible. It can be shown that the solution to this problem is exactly the Principle Component Analysis (PCA) [51] on the set of projection points $\{\mathbf{y}_j\}$. The matrix M is constructed from the first three eigenvectors of the covariance matrix of $\{\mathbf{y}_j\}$. Therefore, the proposed perspective warping function is $F(\mathbf{x}) = MG(\mathbf{x})$, as shown Fig. 3. In our implementation, we perform a post normalization on $F(\mathbf{x})$ to make the resulting points $\{\mathbf{z}_j\}$ in the warp space located around the origin, which is introduced in details in the supplementary material.

Intuition of $F(\mathbf{x})$. $F(\mathbf{x})$ maps the region S in the original space to a region around the origin of the warp space. Fig. 4 shows the perspective warping with different angles between two neighboring cameras. As we can see, when the angle θ is small, the space is squashed more on the far

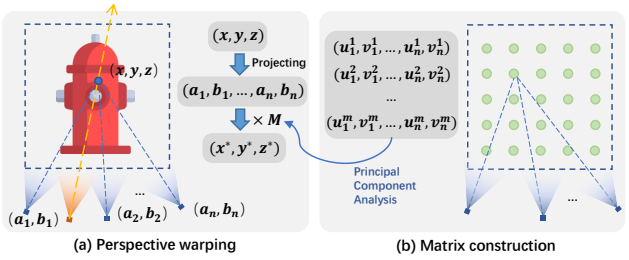

Figure 3. **Our perspective warping method.**

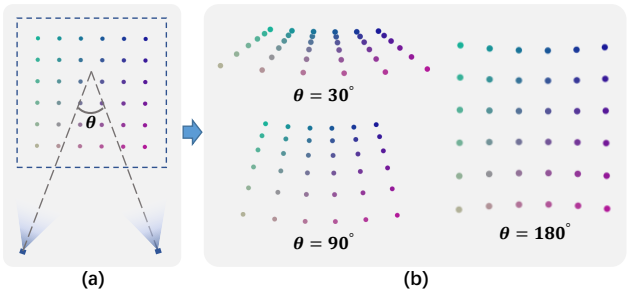

Figure 4. **Visualization of the effect of perspective warping.** (a) Points in the original Euclidean space. (b) Points in the warp space and the corresponding camera angles.

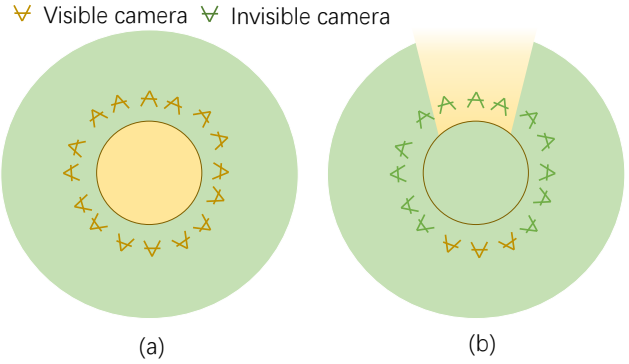

Figure 5. **Inverse sphere warping**. (a) The inner unit sphere are visible to all cameras around. Thus, the warp space is a Euclidean space, which corresponds to the 180° case of perspective warping in Fig. 4. (b) The outer space is only visible to a part of cameras. The outer space uses an NDC warping, which corresponds to the 30° case of perspective warping in Fig. 4.

region, which is a similar behavior to the NDC warping. In this case, the perspective warping reduces to the NDC warping if all the cameras are forward-facing. When the angle becomes larger, the warp space is more similar to the original Euclidean space.

Relationship with inverse sphere warping. We empirically show that inverse sphere warping [3, 61] is also a handcrafted approximation of our perspective warping function. As shown in Fig. 5 (a), the warp space of the inverse sphere warping for the inner unit sphere is simply the original Euclidean space because all the cameras around are visible to this unit sphere, which corresponds to the 180° case in Fig. 4 of the perspective warping. The outer space (Fig. 5 (b)) is only visible from a few far cameras and thus is warped similarly as the NDC warping, which corresponds to the 30° case in Fig. 4 of the perspective warping.

3.3. Space subdivision

In order to apply the perspective warping function $F(\mathbf{x})$, we need to specify n_c cameras onto which we project the point $\mathbf{x}$. According to Definition 1, the properness of the warping function is a local property which only relates the

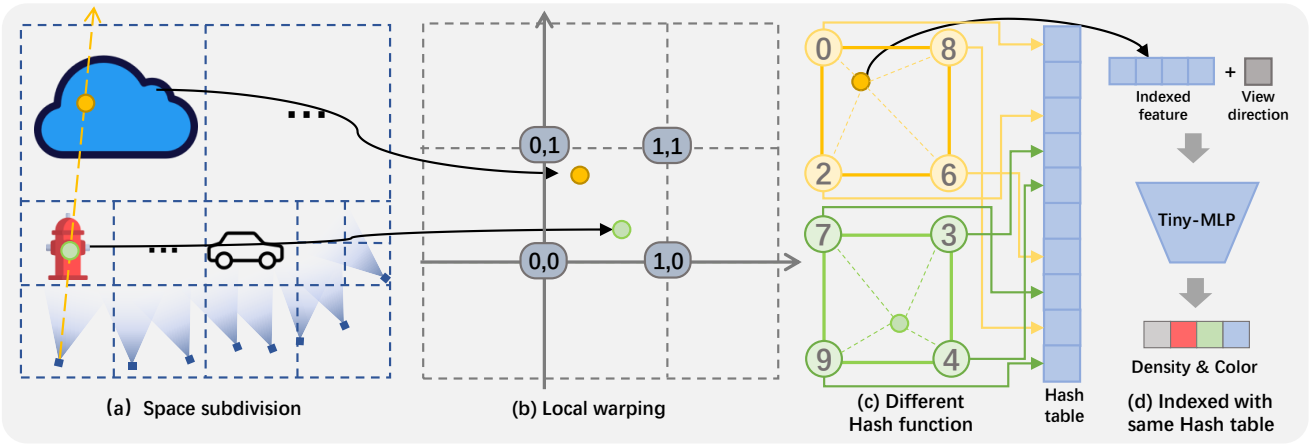

Figure 6. **Pipeline of F^2-NeRF .** (a) Given a large region of interest, we subdivide the space according to the input view frustums. (b) For each sub-region, we construct a perspective warping function based on the visible cameras. The densities and colors are decoded from the scene feature vectors fetched from the same hash table (d) but using different hash functions (c).

visible cameras for the point $\mathbf{x}$. However, for a free trajectory like Fig. 1 (c), visible cameras for different regions are different. This motivates us to adaptively subdivide the space into different regions such that the visible cameras used in $F(\mathbf{x})$ are the same inside a region but are different across regions. In this case, in each subdivided region S_i, a warping function $F_i(\mathbf{x})$ is applied to map S_i to the warp space.

Subdivision strategy. We adopt an octree data structure to store the subdivided regions, which enables us to quickly search for a region and retrieve visible cameras. To construct the octree, we begin with an extremely large bounding box as the root node. Here the box size is 512 times the bounding box size of all camera centers, which is able to contain extremely far-away sky or other objects. Then, starting from the octree root node, we perform a check-and-subdivide procedure. Specifically, on a tree node with s as the side length, we retrieve all visible cameras whose view frustums intersect with this node. Then, if there is any visible camera center whose distance d to the node center is $d \leq \lambda s$, where λ is preset to 3, the node is subdivided into 8 child nodes with side length of $s/2$. Otherwise, the current node is small enough and we stop subdividing it and mark it as a leaf node. For each subdivided child node, we further check the distance and repeat this procedure until we get all n_l leaf nodes $\{S_i | i = 1, 2, ..., n_l\}$ as shown in Fig. 6 (a). Each leaf node is treated as the region S in Problem 1, and for those regions that are visible by more than $n_c = 4$ cameras, we further select n_c visible cameras by making the minimal pairwise distance of the selected cameras as large as possible. A more detailed description of the camera selection strategy can be found in the supplementary material. By applying the warping function F_i, each leaf node is mapped to a region around the origin of its warp space.

3.4. Scene representation

In this section, we will introduce how to build our grid-based scene representation on the warp space, which allows color and density computation for a given point in the warp space. Since the warping functions are different for different leaf nodes, we actually have n_l different warp spaces. A naive solution would be to build n_l different grid representations on each warp space. However, this would cause the number of parameters to grow with the number of leaf nodes. To limit parameter number, we suppose that all warping functions map different leaf nodes to the same warp space and build a hash-grid representation [25] on the warp space with multiple hash functions as shown in Fig. 6.

Hash grid with multiple hash functions. Sharing the same warp space for different leaf nodes will inevitably lead to conflicts, which means two different points in two leaf nodes with different densities and colors are mapped to the same point in the warp space. In the original Instant-NGP [25], there is only one hash function to compute hash values for grid vertices. Here, we use different hash functions for different leaf nodes to alleviate the conflict problem. Specifically, for a point $\mathbf{x}$ in the i-th leaf node, we map it to $\mathbf{z} = F_i(\mathbf{x})$ in the warp space and find z's eight neighboring grid vertices $\hat{\mathbf{v}}$ with integer coordinates. Then, we compute a hash value for each vertex $\hat{\mathbf{v}}$ by a hash function conditioned on the leaf node index i as follows

$$\text{Hash}_i(\hat{\mathbf{v}}) = \left(\bigoplus_{k=1}^{3} \hat{\mathbf{v}}_k \pi_{i,k} + \Delta_{i,k} \right) \mod L, \quad (2)$$

where $\bigoplus$ denotes the bitwise xor operation, and both $\{\pi_{i,k}\}$ and $\{\Delta_{i,k}\}$ are random large prime numbers, which are fixed for a specific leaf node, $k = 1, 2, 3$ means the index of x, y, z coordinate of the warp space, L is the length of the hash table. The computed hash value will be used in indexing the hash

table to retrieve a feature vector for the vertex $\hat{\mathbf{v}}$ and then the feature vector of the point z is trilinearly-interpolated from 8 vertex feature vectors. Finally, the feature vector of z and the view direction $\mathbf{d}$ are fed into a tiny MLP network to produce color and density for the point z, as shown in Fig. 6 (d).

Intuition of Eq. 2. Using different numbers $\pi_{i,k}$ and different offsets $\Delta_{i,k}$ leads to different hash functions in Eq. 2. We use an example in Fig. 6 to show how Eq. 2 works: The green point and the yellow point in two different leaf nodes (Fig. 6 (a)) are mapped by two different warping functions to the same warp space (Fig. 6 (b)). Suppose that both points reside in the same voxel of the warp space. Although the coordinates of their neighboring vertices are the same, the two points will use different hash functions Eq. 2 to compute different hash values for these neighboring vertices (Fig. 6 (c)). Then, the computed hash values will be used in indexing the same hash table to retrieve feature vectors (Fig. 6 (d)). In this case, two points from different leaf nodes share the same neighboring vertices in the warp space but retrieve different vertex feature vectors, which greatly reduces the probability of conflicts. Though some conflicts still remain, they can naturally be resolved by the tiny MLP during the optimization process as observed by Instant-NGP [25].

3.5. Perspective sampling

A proper warping function F also gives us a guideline for sampling points on rays in volume rendering. According to Definition 1, the distance between two points in the proper warp space equals the sum of distances between two projected points on image planes. In this case, by uniformly sampling the points in the warp space, we get a non-uniform sampling in the original Euclidean space but an approximately uniform sampling on images, which improves the sampling efficiency and brings more stable convergence. Specifically, considering a sample point $\mathbf{x}_i = \mathbf{o} + t_i\mathbf{d}$ where $\mathbf{o}, \mathbf{d}$ are the camera origin and direction respectively, we first compute the Jacobian matrix $J_i \in \mathbb{R}^{3\times3}$ of the perspective warping function F at $\mathbf{x}_i$. Then, the next sample point $\mathbf{x}_{i+1} = \mathbf{x}_i + \frac{l}{\|J_i\mathbf{d}\|_2}\mathbf{d}$, where l is a preset parameter controlling the sampling interval and we make a linear approximation here as discussed in the supplementary material.

3.6. Rendering with perspective warping

Based on the descriptions above, we now summarize the rendering procedure in two stages: the preparation stage and the actual rendering stage. (1) In the preparation stage, we subdivide the original space according to the view frustums of the cameras (Sec. 3.3), and construct the local warping functions based on the selected cameras for each sub-region (Sec. 3.2). (2) In the actual rendering stage, we follow the framework of volume rendering to render a pixel color, by sampling points on the camera ray (Sec. 3.5) and conducting

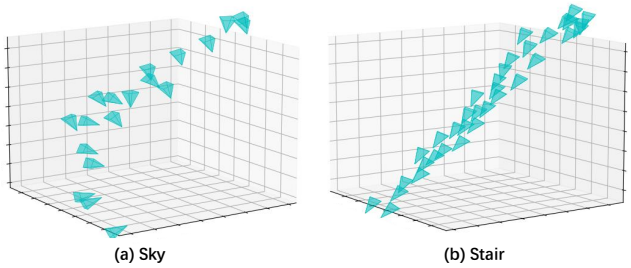

Figure 7. **Trajectories of "Sky" and "Stair" in the Free dataset.**

weighted accumulation of the sampled colors (Sec. 3.1). The sampled densities and colors are fetched from the multi-resolution hash grid (Sec. 3.4).

3.7. Training

The training loss is defined as

$$\mathcal{L} = \mathcal{L}_{recon(c(r),c_{\text{gt}})} + \lambda_{\text{Disp}}\mathcal{L}_{\text{Disp}} + \lambda_{\text{TV}}\mathcal{L}_{\text{TV}}, \quad (3)$$

where $\mathcal{L}_{recon(c(r),c_{\text{gt}})} = \sqrt{(c(r) - c_{\text{gt}})^2 + \epsilon}$ is a color reconstruction loss [3] with $\epsilon = 10^{-4}$. $\mathcal{L}_{\text{Disp}}$ and $\mathcal{L}_{\text{TV}}$ are two regularization losses. The first is a disparity loss $\mathcal{L}_{\text{Disp}}$ that encourages the disparity (inverse depth) not excessively large, which is useful to reduce the floating artifacts. The second is a total variance loss [30] L_{TV} that encourages the points at the borders of two neighboring octree nodes i, j to have similar densities and colors. For all losses, we provide more details in the supplementary material.

4. Experiments

4.1. Experimental Settings

Datasets. We use three datasets for our evaluation. (1) A new unbounded dataset with free trajectories that we collected (called the *Free dataset*). The Free dataset contains seven scenes. Each scene has a narrow and long input camera trajectory and multiple focused foreground objects, which is extremely challenging to build a neural representation for the NVS task. Two of these trajectories are shown in Fig. 7; (2) LLFF dataset [23], which contains eight real unbounded forward-facing scenes with complex geometries; and (3) NeRF-360-V2 dataset [3], which contains seven unbounded 360-degree inward-facing outdoor and indoor scenes. For all the datasets, we follow the commonly-adopted settings that set one of every eight images as testing images and the other as the training set. We use three metrics, PSNR, SSIM, and $\text{LPIPS}_{\text{VGG}}$, for evaluation.

Baselines. We compare F^2-NeRF with the state-of-the-art fast NeRF training methods, the voxel-based methods (1) DVGO [38], (2) Plenoxels [57] and the hash-grid based method (3) Instant-NGP [25]. We also report the results of MLP-based NeRF methods including NeRF++ [61], mip-NeRF [2], and mip-NeRF-360 [3]. Note that both F^2-NeRF

Method	Tr. time	PSNR↑	SSIM↑	LPIPS(VGG)↓
NeRF++ [61]	hours	23.47	0.603	0.499
mip-NeRF-360 [3]	hours	**27.01**	**0.766**	**0.295**
mip-NeRF-360$_{\text{short}}$	30m	22.04	0.537	0.586
Plenoxels [57]	25m	19.13	0.507	0.543
DVGO [38]	21m	23.90	0.651	0.455
Instant-NGP [25]	6m	24.43	0.677	0.413
F^2-NeRF	12m	**26.32**	**0.779**	**0.276**

Table 1. **Results on the Free dataset**. In mip-NeRF-360$_{\text{short}}$, we early stop the training to make them finished in 30 minutes. Training times are evaluated on a 2080ti GPU.

and Instant-NGP [25] are trained for 20k steps with the same batch size but we adopt a LibTorch [28] implementation while Instant-NGP uses a CUDA implementation, which is faster (6min) than ours (13min). All training times are evaluated on a single 2080Ti GPU. Implementation details of F^2-NeRF are included in the supplementary material.

Warping functions. Different warping functions are adopted for baseline methods on different datasets. On the LLFF dataset, all baselines use the NDC warping function, and on both the Free dataset and the NeRF-360-V2 dataset, all baseline methods use the inverse sphere warping function, except Instant-NGP. In the Instant-NGP, we follow the official implementation to enlarge the ray marching bounding box to represent backgrounds and carefully tune the scale parameters on different scenes to achieve the best performance. In comparison, F^2-NeRF always uses the perspective warping for all datasets.

4.2. Comparative studies

We report the quantitative comparisons on the Free dataset in Table 1. F^2-NeRF achieves the best rendering quality among all the fast-training NeRFs. The results for qualitative comparison are shown in Fig. 8. The synthesized images of DVGO [38] and Plenoxels [58] are blurred due to their limited resolutions to represent such a long trajectory. The results of Instant-NGP look sharper but are not clear enough due to its unbalanced scene space organization. In comparison, F^2-NeRF takes advantage of the perspective warping and the adaptive space subdivision to fully exploit the representation capacity, which enables F^2-NeRF to produce better rendering quality. Meanwhile, we find that training a mip-NeRF-360 on the Free dataset for a long time is also able to render clear images. The reason is that during the training process, the large MLP networks used by mip-NeRF-360 are able to gradually concentrate on foreground objects and adaptively allocate more capacity to these foreground objects. However, these MLP networks have to spend a long training time for convergence on the Free Trajectory dataset. With a short training time (30 minutes), the results of mip-NeRF-360 contain many foggy artifacts.

We also evaluate our method on the widely-used unbounded forward-facing dataset (LLFF) and 360° object-centric dataset (NeRF-360-V2) to show the compatibility of the perspective warping with these two kinds of specialized camera trajectories. On both datasets, F^2-NeRF achieves comparable results to the other fast NeRF methods. Note these baseline fast NeRF methods adopt the specially-designed NDC warping or inverse sphere warping for the LLFF dataset or the NeRF-360-V2 dataset while F^2-NeRF always uses the same perspective warping for all datasets.

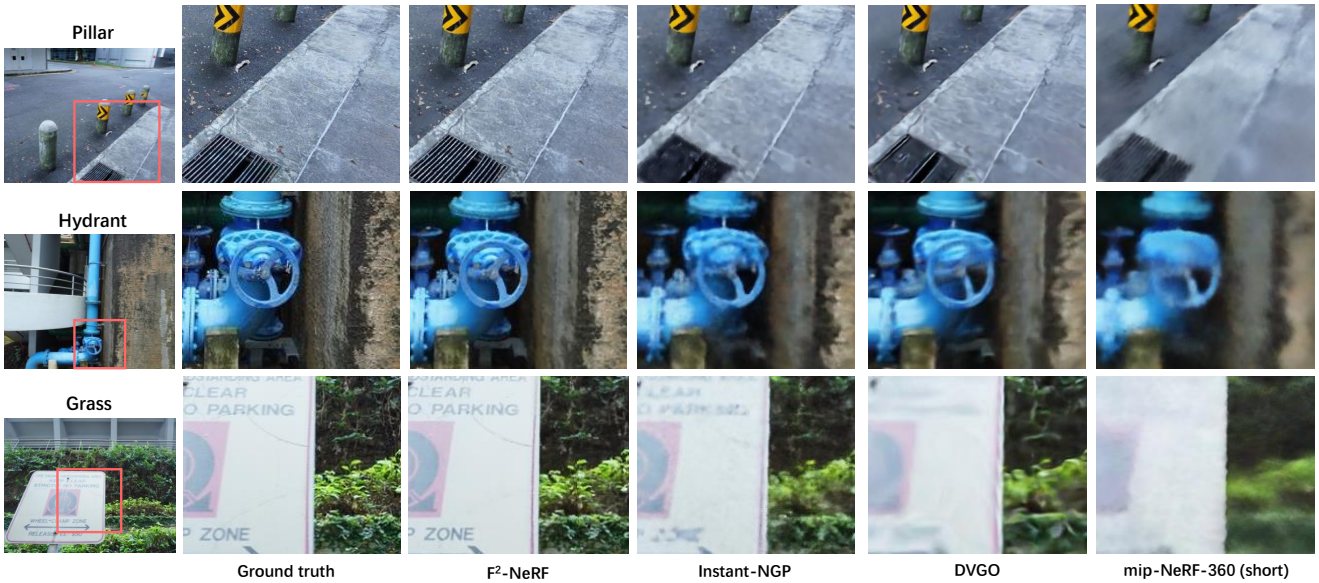

Figure 8. **Visual comparison on the Free dataset.**

Method	Tr. time	PSNR↑	SSIM↑	LPIPS(VGG)↓
NeRF++ [61]	hours	26.21	0.729	0.348
mip-NeRF-360 [2]	hours	**28.94**	**0.837**	**0.208**
Plenoxels [57]	22m	23.35	0.651	0.471
DVGO [38]	16m	25.42	0.695	0.429
Instant-NGP [25]	6m	26.24	0.716	0.404
F^2-NeRF	14m	**26.39**	**0.746**	**0.361**

Table 2. **Results on the NeRF-360-V2 dataset.**

Method	Tr. time	PSNR↑	SSIM↑	LPIPS(VGG)↓
NeRF [24]	hours	26.50	0.811	0.250
mip-NeRF [2]	hours	**26.60**	**0.814**	**0.246**
Plenoxels [57]	17m	26.29	0.839	0.210
DVGO [38]	11m	26.34	0.838	0.197
TensoRF [6]	48m	**26.73**	0.839	0.204
Instant-NGP [25]	6m	25.09	0.758	0.267
F^2-NeRF	13m	26.54	**0.844**	**0.189**

Table 3. **Results on the LLFF dataset.**

This demonstrates the compatibility of the perspective warping with different trajectories.

4.3. Ablation studies

We conduct ablation studies on the "pillar" from the Free dataset. In the ablation studies, we use the multi-resolution hash grid [25] as the scene representation and change the warping functions and the sampling strategies. The warping functions include the inverse sphere warping (Inv. warp), the perspective warping (Pers. warp), and no warping (w/o warp). In the implementation of inverse sphere warping on the Free dataset, we use the bounding sphere of all camera positions as the foreground inner sphere and treat the space outside the sphere as backgrounds. For the point sampling strategies, we consider sampling by disparity (inverse-depth) (Disp. Sampling) used in mip-NeRF-360 [3], sampling by the exponential function (Exp. Sampling) used in Instant-NGP [25], and our perspective sampling (Sec. 3.5). The quantitative results are shown in Table 4 and some qualitative results are shown in Fig. 9.

As shown in Table 4, the basic model (A) without space warping and using disparity sampling performs worst. The model (B) with Inv. warping improves the performances, which shows better compatibility with unbounded scenes compared with no warping (A). The model (C) replaces the disparity sampling with the exponential sampling, which makes the results better. The model (D) uses the proposed perspective warping, whose performance increases drastically compared to the inverse sphere warping. This demonstrates the effectiveness of our perspective warping on free trajectories. The model (E) further applies perspective sampling, which produces the best performance.

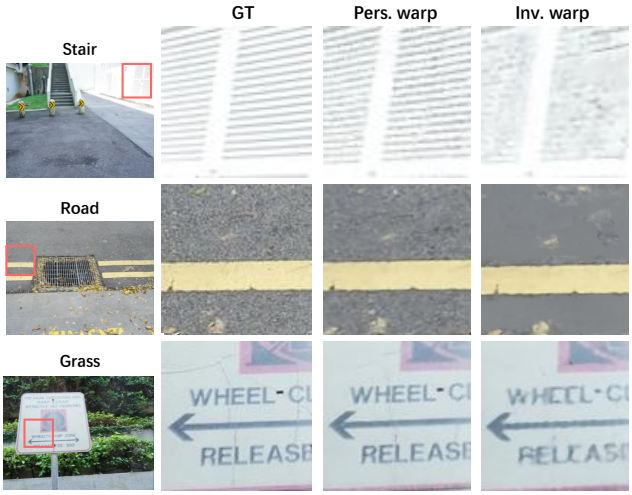

Figure 9. **Visual comparison of different warping techniques.** Our perspective warping renders more clear images than inverse sphere warping [3].

Setting	PSNR↑	SSIM↑	LPIPS↓
A) w/o warp + Disp. sample	26.81	0.693	0.385
B) Inv. warp + Disp. sample	27.09	0.711	0.358
C) Inv. warp + Exp. sample	27.86	0.751	0.289
D) Pers. warp + Exp. sample	28.65	0.796	0.221
E) Pers. warp + Pers. sample	**28.76**	**0.798**	**0.219**

Table 4. **Ablation study on the "pillar" scene of the Free dataset.** F^2-NeRF with perspective warping and sampling achieves the best quantitative results.

5. Conclusion

In the NVS task of unbounded scenes, previous NeRF methods mainly rely on the NDC warping or the inverse sphere warping to process the forward-facing camera trajectory or the 360° object-centric trajectory. In this paper, we conduct an in-depth analysis of the space warping function and propose a novel perspective warping, which is able to handle arbitrary input camera trajectories. Based on the perspective warping, we develop a novel F^2-NeRF in the grid-based NeRF framework. Extensive experiments demonstrate that the proposed F^2-NeRF is able to render high-quality images with arbitrary trajectories while only requiring a few minutes of training time.

Potential negative societal impact. F^2-NeRF can be possibly used for misleading fake image generation.

Acknowledgement. This study is supported by the Ministry of Education, Singapore, under its MOE AcRF Tier 2 (MOE-T2EP20221-0012), NTU NAP, and under the RIE2020 Industry Alignment Fund – Industry Collaboration Projects (IAF-ICP) Funding Initiative, as well as cash and in-kind contribution from the industry partner(s). Lingjie Liu and Christian Theobalt have been supported by the ERC Consolidator Grant 4DReply (770784).

References

[1] Kara-Ali Aliev, Artem Sevastopolsky, Maria Kolos, Dmitry Ulyanov, and Victor Lempitsky. Neural point-based graphics. In *ECCV*, 2020. 2

[2] Jonathan T. Barron, Ben Mildenhall, Matthew Tancik, Peter Hedman, Ricardo Martin-Brualla, and Pratul P. Srinivasan. Mip-nerf: A multiscale representation for anti-aliasing neural radiance fields. In *ICCV*, 2021. 2, 3, 6, 8

[3] Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Mip-nerf 360: Unbounded anti-aliased neural radiance fields. In *CVPR*, 2022. 2, 3, 4, 6, 7, 8

[4] Mark Boss, Raphael Braun, Varun Jampani, Jonathan T. Barron, Ce Liu, and Hendrik P. A. Lensch. Nerd: Neural reflectance decomposition from image collections. In *ICCV*, 2021. 2

[5] Chris Buehler, Michael Bosse, Leonard McMillan, Steven J. Gortler, and Michael F. Cohen. Unstructured lumigraph rendering. In *SIGGRAPH*, 2001. 2

[6] Anpei Chen, Zexiang Xu, Andreas Geiger, Jingyi Yu, and Hao Su. Tensorf: Tensorial radiance fields. In *ECCV*, 2022. 1, 2, 3, 8

[7] Anpei Chen, Zexiang Xu, Fuqiang Zhao, Xiaoshuai Zhang, Fanbo Xiang, Jingyi Yu, and Hao Su. Mvsnerf: Fast generalizable radiance field reconstruction from multi-view stereo. In *ICCV*, 2021. 2

[8] Abe Davis, Marc Levoy, and Frédo Durand. Unstructured light fields. *Comput. Graph. Forum*, 2012. 2

[9] Paul E. Debevec, Camillo J. Taylor, and Jitendra Malik. Modeling and rendering architecture from photographs: A hybrid geometry- and image-based approach. In *SIGGRAPH*, 1996. 2

[10] Helisa Dhamo, Keisuke Tateno, Iro Laina, Nassir Navab, and Federico Tombari. Peeking behind objects: Layered depth prediction from a single image. *Pattern Recognit. Lett.*, 2019. 2

[11] Rizal Fathony, Anit Kumar Sahu, Devin Willmott, and J Zico Kolter. Multiplicative filter networks. In *International Conference on Learning Representations*, 2020. 2

[12] John Flynn, Michael Broxton, Paul E. Debevec, Matthew DuVall, Graham Fyffe, Ryan S. Overbeck, Noah Snavely, and Richard Tucker. Deepview: View synthesis with learned gradient descent. In *CVPR*, 2019. 2

[13] Steven J. Gortler, Radek Grzeszczuk, Richard Szeliski, and Michael F. Cohen. The lumigraph. In *SIGGRAPH*, 1996. 2

[14] Tong He, John P. Collomosse, Hailin Jin, and Stefano Soatto. Deepvoxels++: Enhancing the fidelity of novel view synthesis from 3d voxel embeddings. In *ACCV*, 2020. 2

[15] Anat Levin and Frédo Durand. Linear view synthesis using a dimensionality gap light field prior. In *CVPR*, 2010. 2

[16] Zhengqi Li, Wenqi Xian, Abe Davis, and Noah Snavely. Crowdsampling the plenoptic function. In *ECCV*, 2020. 2

[17] David B Lindell, Dave Van Veen, Jeong Joon Park, and Gordon Wetzstein. Bacon: Band-limited coordinate networks for multiscale scene representation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16252–16262, 2022. 2

[18] Lingjie Liu, Jiatao Gu, Kyaw Zaw Lin, Tat-Seng Chua, and Christian Theobalt. Neural sparse voxel fields. In *NeurIPS*, 2020. 2

[19] Yuan Liu, Sida Peng, Lingjie Liu, Qianqian Wang, Peng Wang, Christian Theobalt, Xiaowei Zhou, and Wenping Wang. Neural rays for occlusion-aware image-based rendering. *arxiv CS.CV 2107.13421*, 2021. 2

[20] Stephen Lombardi, Tomas Simon, Jason M. Saragih, Gabriel Schwartz, Andreas M. Lehrmann, and Yaser Sheikh. Neural volumes: learning dynamic renderable volumes from images. *ACM Trans. Graph.*, 2019. 2

[21] Stephen Lombardi, Tomas Simon, Gabriel Schwartz, Michael Zollhöfer, Yaser Sheikh, and Jason M. Saragih. Mixture of volumetric primitives for efficient neural rendering. *ACM Trans. Graph.*, 2021. 2

[22] Julien NP Martel, David B Lindell, Connor Z Lin, Eric R Chan, Marco Monteiro, and Gordon Wetzstein. Acorn: Adaptive coordinate networks for neural scene representation. *arXiv preprint arXiv:2105.02788*, 2021. 2

[23] Ben Mildenhall, Pratul P. Srinivasan, Rodrigo Ortiz Cayon, Nima Khademi Kalantari, Ravi Ramamoorthi, Ren Ng, and Abhishek Kar. Local light field fusion: practical view synthesis with prescriptive sampling guidelines. *ACM Trans. Graph.*, 2019. 2, 6

[24] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. In *ECCV*, 2020. 1, 2, 3, 8

[25] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Trans. Graph.*, 2022. 1, 2, 3, 5, 6, 7, 8

[26] Jacob Munkberg, Jon Hasselgren, Tianchang Shen, Jun Gao, Wenzheng Chen, Alex Evans, Thomas Müller, and Sanja Fidler. Extracting triangular 3d models, materials, and lighting from images. In *CVPR*, 2022. 2

[27] Michael Oechsle, Songyou Peng, and Andreas Geiger. UNISURF: unifying neural implicit surfaces and radiance fields for multi-view reconstruction. In *ICCV*, 2021. 2

[28] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019. 7

[29] Sameera Ramasinghe and Simon Lucey. Beyond periodicity: Towards a unifying framework for activations in coordinate-mlps. In *European Conference on Computer Vision*, pages 142–158. Springer, 2022. 2

[30] Leonid I Rudin and Stanley Osher. Total variation based image restoration with free local constraints. In *Proceedings of 1st international conference on image processing*, volume 1, pages 31–35. IEEE, 1994. 6

[31] Jonathan Shade, Steven J. Gortler, Li-wei He, and Richard Szeliski. Layered depth images. In *SIGGRAPH*, 1998. 2

[32] Meng-Li Shih, Shih-Yang Su, Johannes Kopf, and Jia-Bin Huang. 3d photography using context-aware layered depth inpainting. In *CVPR*, 2020. 2

[33] Vincent Sitzmann, Julien Martel, Alexander Bergman, David Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. *Advances in Neural Information Processing Systems*, 33:7462–7473, 2020. 2

[34] Vincent Sitzmann, Justus Thies, Felix Heide, Matthias Nießner, Gordon Wetzstein, and Michael Zollhöfer. Deepvoxels: Learning persistent 3d feature embeddings. In *CVPR*, 2019. 2

[35] Vincent Sitzmann, Michael Zollhöfer, and Gordon Wetzstein. Scene representation networks: Continuous 3d-structure-aware neural scene representations. In *NeurIPS*, 2019. 2

[36] Pratul P. Srinivasan, Richard Tucker, Jonathan T. Barron, Ravi Ramamoorthi, Ren Ng, and Noah Snavely. Pushing the boundaries of view extrapolation with multiplane images. In *CVPR*, 2019. 2

[37] Mohammed Suhail, Carlos Esteves, Leonid Sigal, and Ameesh Makadia. Generalizable patch-based neural rendering. *arXiv preprint arXiv:2207.10662*, 2022. 2

[38] Cheng Sun, Min Sun, and Hwann-Tzong Chen. Direct voxel grid optimization: Super-fast convergence for radiance fields reconstruction. In *CVPR*, 2022. 1, 2, 3, 6, 7, 8

[39] Towaki Takikawa, Joey Litalien, Kangxue Yin, Karsten Kreis, Charles Loop, Derek Nowrouzezahrai, Alec Jacobson, Morgan McGuire, and Sanja Fidler. Neural geometric level of detail: Real-time rendering with implicit 3d shapes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11358–11367, 2021. 2

[40] Matthew Tancik, Vincent Casser, Xinchen Yan, Sabeek Pradhan, Ben Mildenhall, Pratul P Srinivasan, Jonathan T Barron, and Henrik Kretzschmar. Block-nerf: Scalable large scene neural view synthesis. In *CVPR*, 2022. 3

[41] Matthew Tancik, Ethan Weber, Evonne Ng, Ruilong Li, Brent Yi, Justin Kerr, Terrance Wang, Alexander Kristoffersen, Jake Austin, Kamyar Salahi, Abhik Ahuja, David McAllister, and Angjoo Kanazawa. Nerfstudio: A modular framework for neural radiance field development, 2023. 2

[42] Ayush Tewari, Justus Thies, Ben Mildenhall, Pratul Srinivasan, Edgar Tretschk, W Yifan, Christoph Lassner, Vincent Sitzmann, Ricardo Martin-Brualla, Stephen Lombardi, et al. Advances in neural rendering. *Computer Graphics Forum*, 41(2):703–735, 2022. 1, 2

[43] Justus Thies, Michael Zollhöfer, and Matthias Nießner. Deferred neural rendering: image synthesis using neural textures. *ACM Trans. Graph.*, 2019. 2

[44] Richard Tucker and Noah Snavely. Single-view view synthesis with multiplane images. In *CVPR*, 2020. 2

[45] Shubham Tulsiani, Richard Tucker, and Noah Snavely. Layer-structured 3d scene inference via view synthesis. In *ECCV*, 2018. 2

[46] Haithem Turki, Deva Ramanan, and Mahadev Satyanarayanan. Mega-nerf: Scalable construction of large-scale nerfs for virtual fly-throughs. In *CVPR*, 2022. 3

[47] Michael Waechter, Nils Moehrle, and Michael Goesele. Let there be color! large-scale texturing of 3d reconstructions. In *ECCV*, 2014. 2

[48] Dan Wang, Xinrui Cui, Septimiu Salcudean, and Z Jane Wang. Generalizable neural radiance fields for novel view synthesis with transformer. *arXiv preprint arXiv:2206.05375*, 2022. 2

[49] Peng Wang, Lingjie Liu, Yuan Liu, Christian Theobalt, Taku Komura, and Wenping Wang. Neus: Learning neural implicit surfaces by volume rendering for multi-view reconstruction. In *NeurIPS*, 2021. 2

[50] Qianqian Wang, Zhicheng Wang, Kyle Genova, Pratul P. Srinivasan, Howard Zhou, Jonathan T. Barron, Ricardo Martin-Brualla, Noah Snavely, and Thomas A. Funkhouser. Ibrnet: Learning multi-view image-based rendering. In *CVPR*, 2021. 2

[51] Svante Wold, Kim Esbensen, and Paul Geladi. Principal component analysis. *Chemometrics and intelligent laboratory systems*, 2(1-3):37–52, 1987. 2, 4

[52] Daniel N. Wood, Daniel I. Azuma, Ken Aldinger, Brian Curless, Tom Duchamp, David Salesin, and Werner Stuetzle. Surface light fields for 3d photography. In *SIGGRAPH*, 2000. 2

[53] Yuanbo Xiangli, Linning Xu, Xingang Pan, Nanxuan Zhao, Anyi Rao, Christian Theobalt, Bo Dai, and Dahua Lin. Bungeenerf: Progressive neural radiance field for extreme multi-scale scene rendering. In *ECCV*, 2022. 3

[54] Qiangeng Xu, Zexiang Xu, Julien Philip, Sai Bi, Zhixin Shu, Kalyan Sunkavalli, and Ulrich Neumann. Point-nerf: Point-based neural radiance fields. In *CVPR*, 2022. 2

[55] Lior Yariv, Jiatao Gu, Yoni Kasten, and Yaron Lipman. Volume rendering of neural implicit surfaces. *Advances in Neural Information Processing Systems*, 34:4805–4815, 2021. 2

[56] Lior Yariv, Yoni Kasten, Dror Moran, Meirav Galun, Matan Atzmon, Basri Ronen, and Yaron Lipman. Multiview neural surface reconstruction by disentangling geometry and appearance. *NeurIPS*, 2020. 2

[57] Alex Yu, Sara Fridovich-Keil, Matthew Tancik, Qinhong Chen, Benjamin Recht, and Angjoo Kanazawa. Plenoxels: Radiance fields without neural networks. In *CVPR*, 2022. 1, 2, 3, 6, 7, 8

[58] Alex Yu, Ruilong Li, Matthew Tancik, Hao Li, Ren Ng, and Angjoo Kanazawa. Plenoctrees for real-time rendering of neural radiance fields. In *ICCV*, 2021. 7

[59] Alex Yu, Vickie Ye, Matthew Tancik, and Angjoo Kanazawa. pixelnerf: Neural radiance fields from one or few images. In *CVPR*, 2021. 2

[60] Kai Zhang, Fujun Luan, Qianqian Wang, Kavita Bala, and Noah Snavely. Physg: Inverse rendering with spherical gaussians for physics-based material editing and relighting. In *CVPR*, 2021. 2

[61] Kai Zhang, Gernot Riegler, Noah Snavely, and Vladlen Koltun. Nerf++: Analyzing and improving neural radiance fields. *arxiv CS.CV 2010.07492*, 2020. 2, 3, 4, 6, 7, 8

[62] Xiuming Zhang, Pratul P. Srinivasan, Boyang Deng, Paul E. Debevec, William T. Freeman, and Jonathan T. Barron. Nerfactor: Neural factorization of shape and reflectance under an unknown illumination. *arxiv CS.CV 2106.01970*, 2021. 2

[63] Tinghui Zhou, Richard Tucker, John Flynn, Graham Fyffe, and Noah Snavely. Stereo magnification: learning view synthesis using multiplane images. *ACM Trans. Graph.*, 2018. 2